

Actividad - Centralización de Alertas

Fabián Guzmán Villegas, Maycol Silva Orozco, James Machado Hernández

Servicio Nacional de Aprendizaje - SENA

Tecnólogo en Análisis y Desarrollo de Software

Número de grupo o ficha

3172293

Instructor: Edwar Velázquez

Fecha: 21/09/2026



Introducción

El día de hoy nuestro grupo de trabajo buscará implementar los 5 tipos de escenarios de un CRUD, con la información y los ejemplos vistos en clase nos quedó un conocimiento el cual usaremos para poder implementar los siguientes 5 casos que son:

1. Error al crear usuario
2. Confirmación antes de eliminar
3. Eliminación exitosa
4. Cancelación de eliminación
5. Error inesperado del sistema

Desarrollo

1. Error al crear usuario

The screenshot shows a VS Code editor with a project named '3172293-jamf-project'. The file explorer on the left shows a directory structure with folders like 'assets', 'data', 'features', 'auth', 'dishes', 'home', 'inventory', 'menu', 'orders', 'permissions', 'products', 'providers', 'users', 'services', 'shared', 'components', 'layouts', 'schemas', and 'services'. The 'services' folder is expanded, showing 'alertService.js' and 'index.js'. The 'alertService.js' file is open in the editor, showing the following code:

```

src > shared > services > JS alertService.js > showUserErrorAlert
83 export function showOpsAlert({
95   customClass: {
96     confirmButton: "bg-red-600 text-white hover:bg-red-700 px-4 py-2
97     rounded-lg font-medium",
98     footer: "text-sm text-gray-500",
99   },
100   buttonsStyling: false,
101 });
102
103
104
105 // Error al crear usuario
106 export function showUserErrorAlert({
107   title = "Error al crear usuario",
108   text = "No se pudo registrar el usuario. Por favor, verifica que los campos
109   estén correctos.",
110 }) {
111   return Swal.fire({
112     icon: "error",
113     title,
114     text,
115     confirmButtonText: "Intentar de nuevo",
116     customClass: {
117       popup: "rounded-2xl",
118       title: "text-red-600 font-bold",
119       confirmButton: "bg-red-600 text-white hover:bg-red-700 px-4 py-2
120       rounded-lg font-medium transition",
121     },
122     buttonsStyling: false,
123   });
124 }

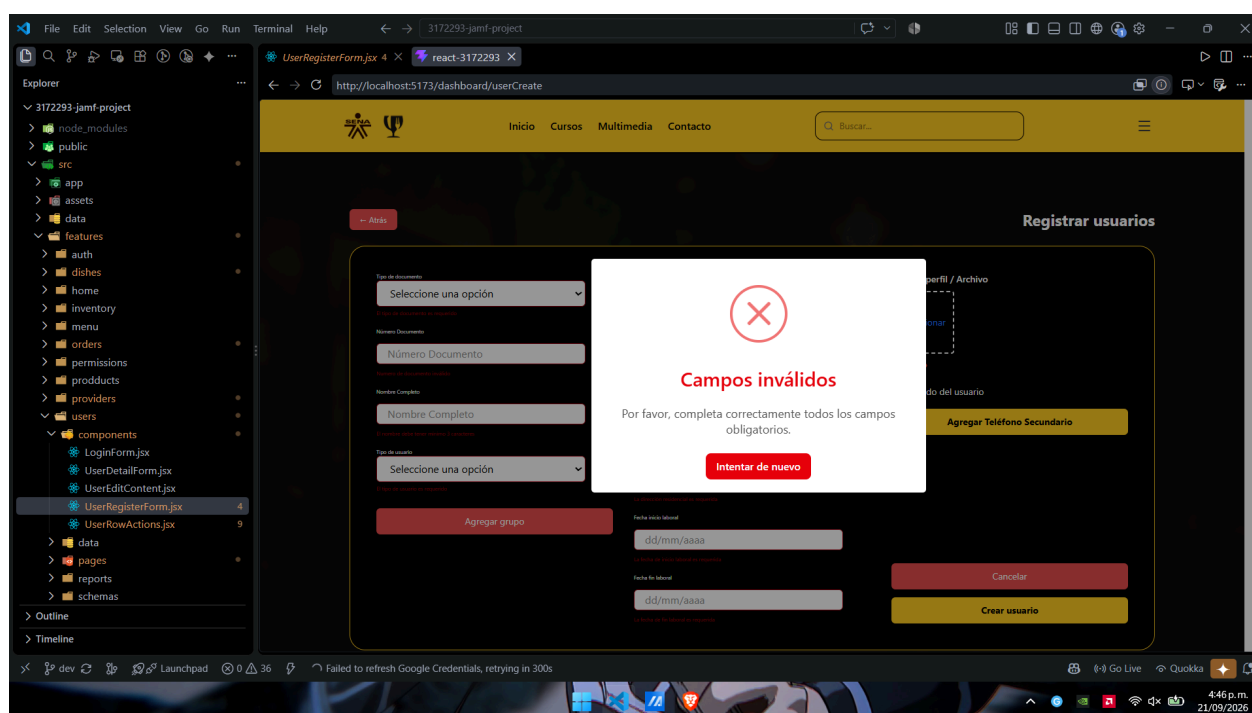
```

The code defines a function `showUserErrorAlert` that uses `Swal.fire` to display an error alert. The alert has a red background, a bold title, and a confirm button labeled "Intentar de nuevo". The alert text says: "No se pudo registrar el usuario. Por favor, verifica que los campos estén correctos.".

```

src > features > users > components > UserRegisterForm.jsx > UserRegisterForm > handleSubmit
14 export default function UserRegisterForm() {
41   const handleChange = (e) => {
43     setData((prev) => ({
46       ...prev,
47     }));
48   });
49   // FIX 1: Marcamos handleSubmit como async para permitir 'await'
50   const handleSubmit = async (e) => {
51     e.preventDefault();
52
53     const result = userSchema.safeParse(formData);
54
55     if (!result.success) {
56       console.log("Errores de Zod:", result.error.issues);
57       const fieldErrors = {};
58       result.error.issues.forEach((issue) => {
59         fieldErrors[issue.path[0]] = issue.message;
60       });
61       setErrors(fieldErrors);
62
63       // ALERTA DE ERROR
64       await showUserErrorAlert({
65         title: "Campos inválidos",
66         text: "Por favor, completa correctamente todos los campos obligatorios.",
67       });
68     }
69     return;
70   }
71   setErrors({});
72
73   try {
74     // FIX 2: Se corrigió la sintaxis de los comentarios de prueba
75     // const response = await createUser(result.data);
76     // console.log("Usuario Creado:", response);
77   }

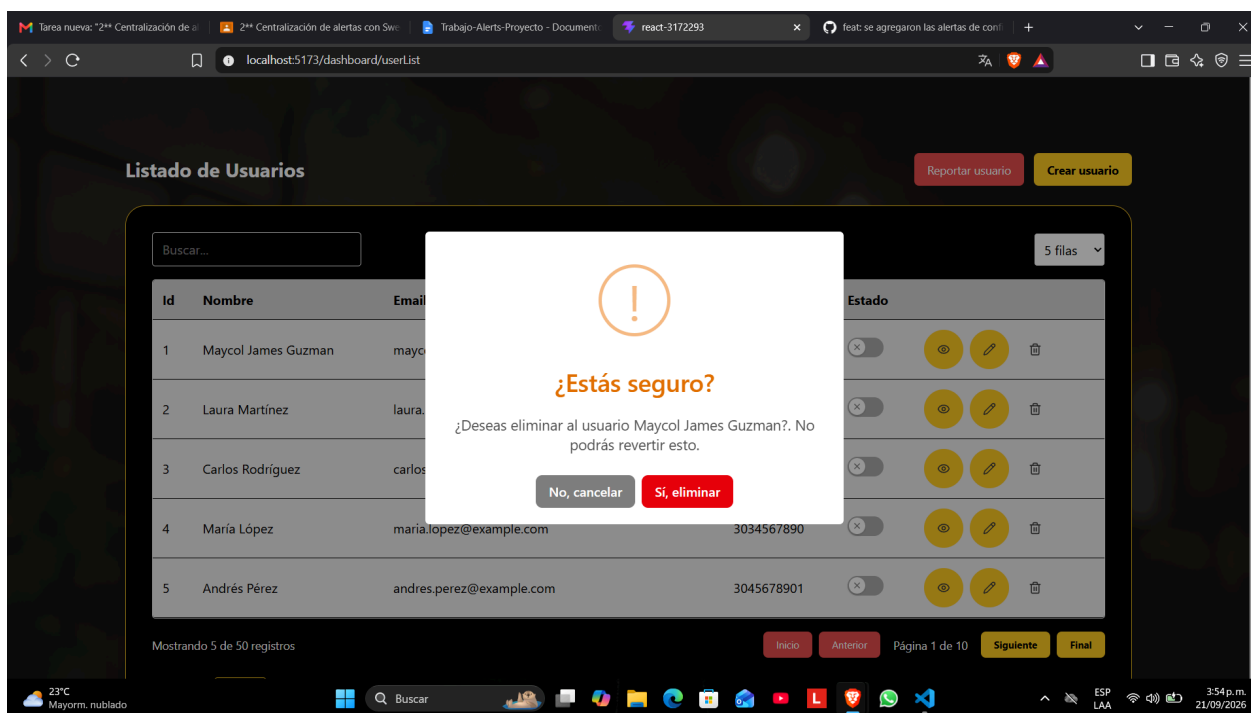
```



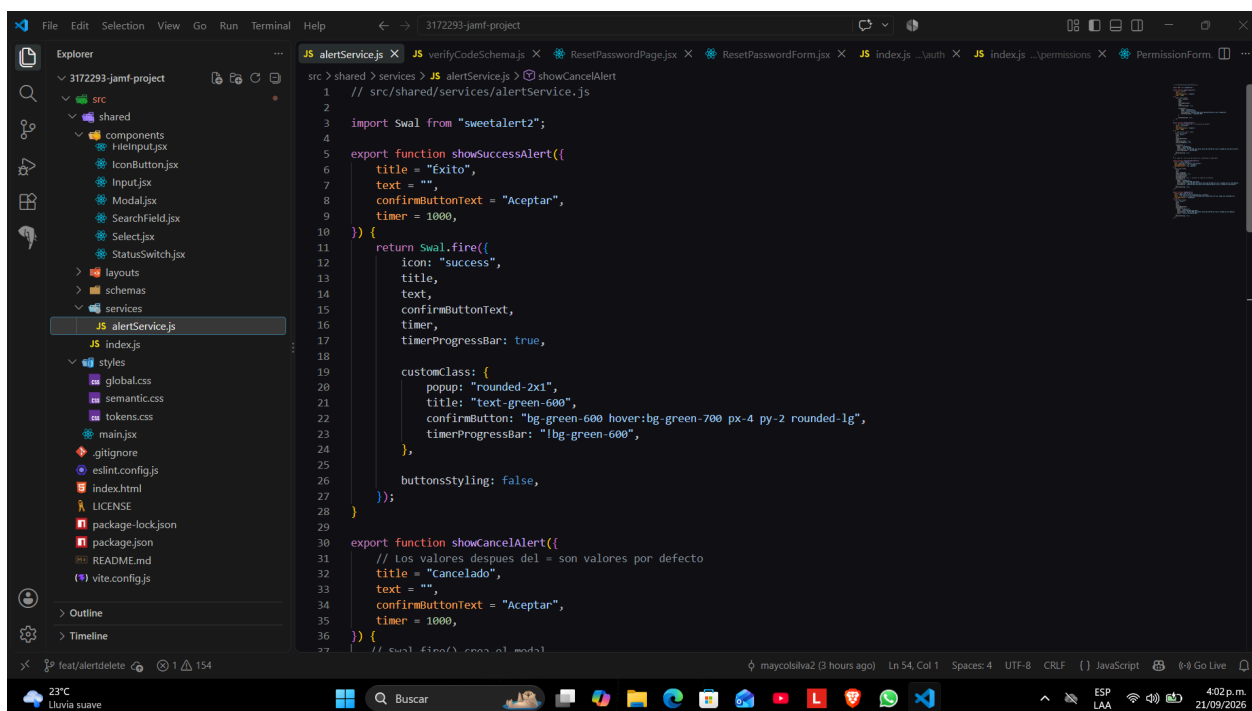
2. Confirmación antes de eliminar

The screenshot shows a code editor with the following components visible:

- File Explorer (Left):** Shows the project structure for '3172293-jamf-project'. The 'users' directory is expanded, showing 'components' and 'UserRowActions.jsx' (which is selected).
- Code Editor (Main):** Displays the code for 'UserRowActions.jsx'. The code includes:
 - Imports for 'navigate' and 'showSuccessAlert'.
 - A function 'handleEdit' that calls 'navigate' to edit a user.
 - A function 'handleDelete' that calls 'showConfirmDeleteAlert' and 'showSuccessAlert'.
 - A function 'handleView' that calls 'showCancelAlert'.
 - Conditional rendering logic for the 'actions' array based on the user's role ('admin' or 'user').
- Terminal (Bottom):** Shows the command 'npm run dev' and the output 'D:\314F (2 months ago) Ln 50, Col 1 Spaces: 2 UTF-8 CRLF {} JavaScript JSX Babel JS Live'.



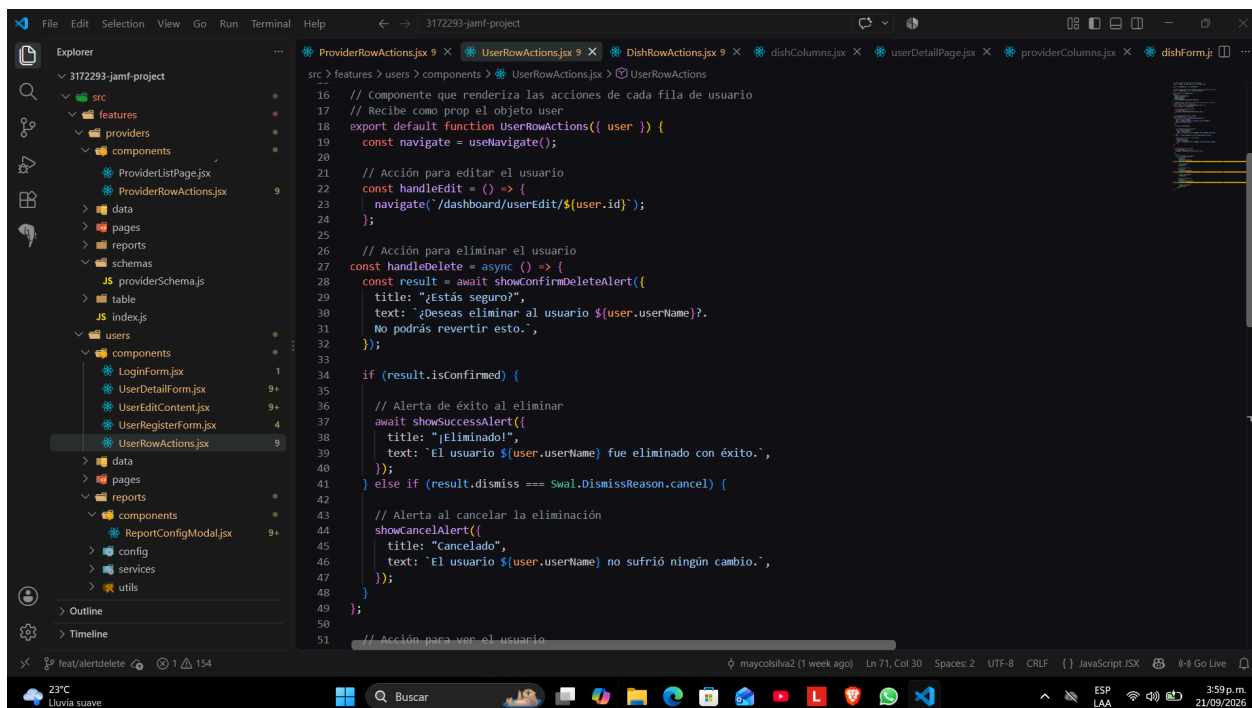
3. Eliminación exitosa



```

1 // src/shared/services/alertService.js
2
3 import Swal from "sweetalert2";
4
5 export function showSuccessAlert({
6   title = "Éxito",
7   text = "",
8   confirmButtonText = "Aceptar",
9   timer = 1000,
10 }) {
11   return Swal.fire({
12     icon: "success",
13     title,
14     text,
15     confirmButtonText,
16     timer,
17     timerProgressBar: true,
18
19     customClass: {
20       popup: "rounded-2xl",
21       title: "text-green-600",
22       confirmButton: "bg-green-600 hover:bg-green-700 px-4 py-2 rounded-lg",
23       timerProgressBar: "bg-green-600",
24     },
25
26     buttonsStyling: false,
27   });
28 }
29
30 export function showCancelAlert({
31   // Los valores después del = son valores por defecto
32   title = "Cancelado",
33   text = "",
34   confirmButtonText = "Aceptar",
35   timer = 1000,
36 }) {
37   // Swal.fire() con el modal

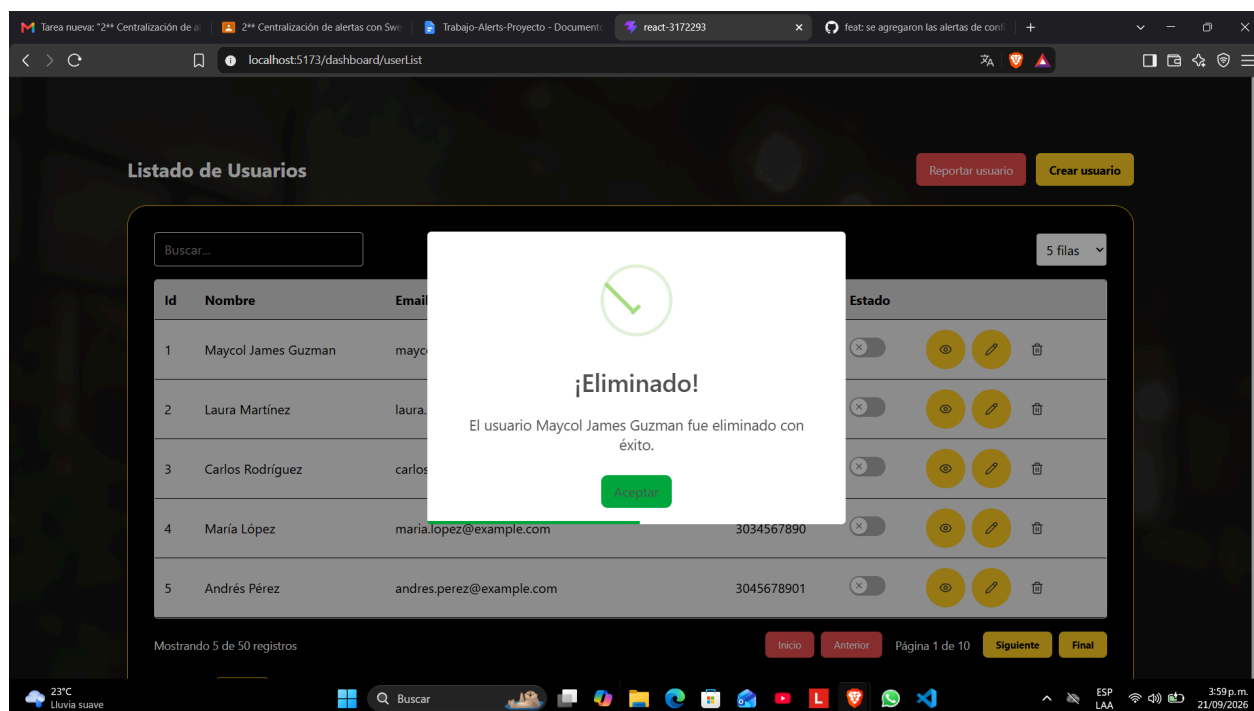
```



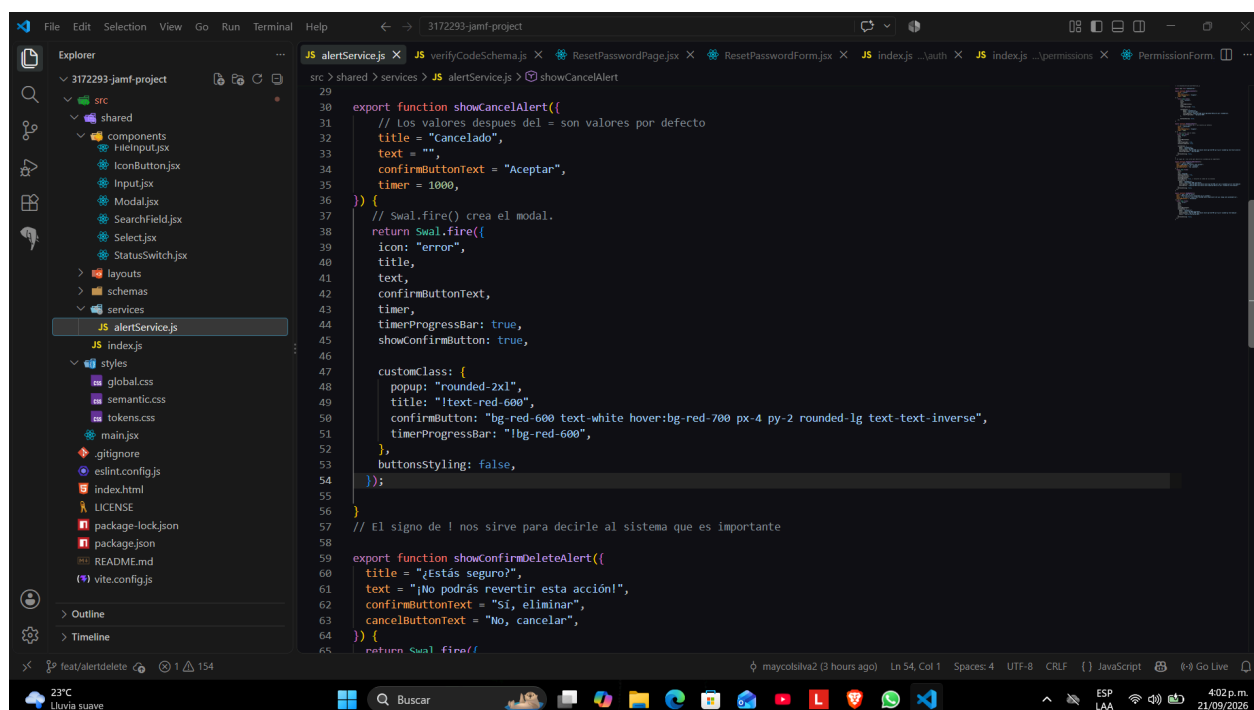
```

16 // Componente que renderiza las acciones de cada fila de usuario
17 // Recibe como prop el objeto user
18 export default function UserRowActions({ user }) {
19   const navigate = useNavigate();
20
21   // Acción para editar el usuario
22   const handleEdit = () => {
23     navigate(`/dashboard/userEdit/${user.id}`);
24   };
25
26   // Acción para eliminar el usuario
27   const handleDelete = async () => {
28     const result = await showConfirmDeleteAlert({
29       title: "¿Estás seguro?",
30       text: "¿Deseas eliminar al usuario ${user.userName}?. No podrás revertir esto.",
31     });
32
33     if (result.isConfirmed) {
34       // Alerta de éxito al eliminar
35       await showSuccessAlert({
36         title: "¡Eliminado!",
37         text: `El usuario ${user.userName} fue eliminado con éxito.`,
38       });
39     } else if (result.dismiss === Swal.DismissReason.cancel) {
40       // Alerta al cancelar la eliminación
41       showCancelAlert({
42         title: "Cancelado",
43         text: `El usuario ${user.userName} no sufrió ningún cambio.`,
44       });
45     }
46   };
47
48   // Acción para ver el usuario

```



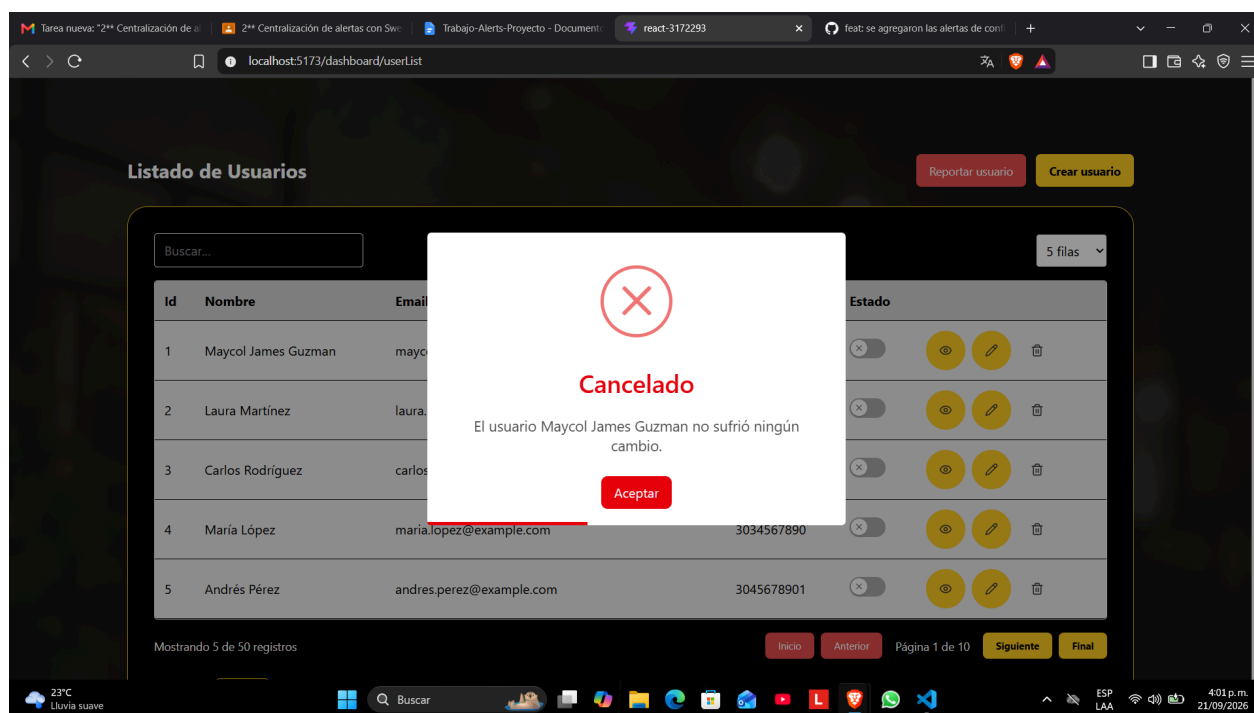
4. Cancelación de eliminación



```

src > features > users > components > UserRowActions.jsx > UserRowActions
16 // Componente que renderiza las acciones de cada fila de usuario
17 // Recibe como prop el objeto user
18 export default function UserRowActions({ user }) {
19   const navigate = useNavigate();
20
21   // Acción para editar el usuario
22   const handleEdit = () => {
23     navigate('/dashboard/userEdit/${user.id}');
24   };
25
26   // Acción para eliminar el usuario
27   const handleDelete = async () => {
28     const result = await showConfirmDeleteAlert({
29       title: "¿Estás seguro?",
30       text: "¿Deseas eliminar al usuario ${user.userName}?. No podrás revertir esto.",
31     });
32
33     if (result.isConfirmed) {
34       // Alerta de éxito al eliminar
35       await showSuccessAlert({
36         title: "¡Eliminado!",
37         text: "El usuario ${user.userName} fue eliminado con éxito.",
38       });
39     } else if (result.dismiss === Swal.DismissReason.cancel) {
40       // Alerta al cancelar la eliminación
41       showCancelAlert({
42         title: "Cancelado",
43         text: "El usuario ${user.userName} no sufrió ningún cambio.",
44       });
45     }
46   };
47
48   // Acción para ver el usuario
49 }
50
51

```



5. Error inesperado del sistema

The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the file `src > shared > services > JS alertService.js`. The code defines a function `showOpsAlert` that uses `Swal.fire` to display an error message. The message text is: "¡Ocurrió un fallo inesperado en el sistema!", with a footer link: "¿Por qué tengo este problema?". The alert has a red background and a white border.

```

82 export function showOpsAlert({
83   title = "Ops, hubo un error",
84   text = "¡Ocurrió un fallo inesperado en el sistema!",
85   footer = 'a href="#" class="text-red-600 hover:underline">¿Por qué tengo este problema?</a>',
86   confirmButtonText = "Entendido",
87 }) = () {
88   return Swal.fire({
89     icon: "error",
90     title,
91     text,
92     footer,
93     confirmButtonText,
94     customClass: {
95       popup: "rounded-2xl",
96       title: "text-red-600 font-bold",
97       confirmButton: "bg-red-600 text-white hover:bg-red-700 px-4 py-2 rounded-lg font-medium",
98       footer: "text-sm text-gray-500",
99     },
100     buttonsStyling: false,
101   });
102 }
103
104

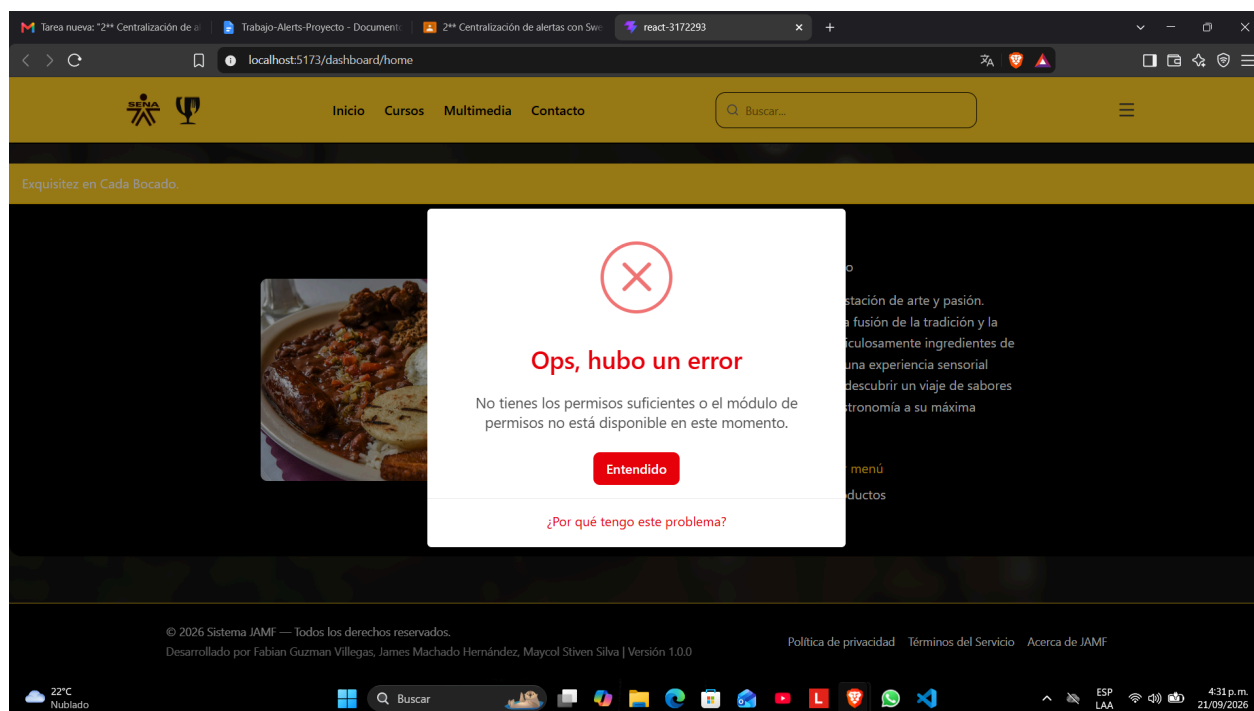
```

The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the file `src > shared > layouts > Navbar.js`. The code defines a function `Navbar` that returns a JSX element representing a navigation bar. The bar includes a main section with links to the dashboard and home, and a hidden section for desktop users with links to the dashboard/home and a button to view courses.

```

28 export default function Navbar() {
29   const handlePermissionsClick = () => {
30     showOpsAlert({
31       title: "Ops, hubo un error",
32       text: "No tienes los permisos suficientes o el módulo de permisos no está disponible en este momento.",
33     });
34   };
35
36   return (
37     <nav className="w-full bg-brand border-b-2 border-black">
38       <div className="mx-auto max-w-7xl px-4 py-2">
39         {/* FILA PRINCIPAL */}
40         <div className="flex items-center justify-between gap-2 h-14 md:h-16">
41           {/* Logos agrupados */}
42           <div>
43             <Link to="/dashboard" className="flex items-center gap-2 sm:gap-4 shrink-0">
44               <img src={logo2} alt="Logo Rico" className="h-7 sm:h-10 object-contain" />
45               <img src={logo} alt="Logo SENA" className="h-7 sm:h-10 object-contain" />
46             </Link>
47           </div>
48           {/* Links de navegación (Solo Desktop) */}
49           <ul className="hidden md:flex items-center gap-6 font-medium">
50             <li>
51               <Link to="/dashboard/home" className="hover:text-primary transition">
52                 Inicio
53               </Link>
54             </li>
55             <li>
56               <button
57                 onClick={handleCursosClick}
58                 className="hover:text-primary transition cursor-pointer font-medium"
59               >
60                 Cursos
61               </button>
62             </li>
63           </ul>
64         </div>
65       </div>
66     </nav>
67   );
68 }

```



Conclusiones:

Al final de esta actividad comprendimos como es de importante una alerta para que el usuario sepa en qué paso va de su proceso en nuestra página, entendemos que la UX para la gratitud de nuestros visitantes es algo que debemos de tener muy presentes y entendimos cómo se implementan de manera correcta las 5 situaciones del CRUD.